



University of Engineering & Technology, Taxila

Faculty of Telecommunication & Information Engineering

B.Sc. Computer Engineering

Final Report

Machine Learning-Lab

(CP-310L)

**Topic: Movie Recommendation System Using Machine
Learning**

Names: Laiqa Tahir Khan, Muhammad Saffi Ul Haq

Roll Nos: 22-CP-02 , 22-CP-78

Course Instructor: Dr. Waqar Ahmad

Lab Instructor: Engr. Mona Waseem

Semester: 6th

Section: Omega

Date: May 06, 2025

Movie Recommender System

Introduction

In the digital era, recommendation systems have emerged as essential tools to help users find relevant content amidst vast quantities of data. Whether it is movies, books, products, or even news articles, recommender systems personalize user experiences by suggesting content tailored to individual preferences. This report focuses on the development of a **Movie Recommender System** using machine learning and natural language processing (NLP) techniques. The aim of the project is to create a system that recommends movies similar to a selected one based on content-based filtering.

The recommender system is developed using the **Streamlit** framework for web deployment, with the core recommendation logic implemented in Python using **pandas**, **scikit-learn**, and **NLTK**. The project consists of multiple files including a Jupyter notebook for initial model development, pickled files for serialized data and similarity scores, and a Streamlit application (app.py) to interface with the user.

Objective

The main objective of this project is to build an efficient and accurate **content-based movie recommender system** that:

- Takes user input for a movie title.
- Suggests similar movies based on textual features like genres, cast, director, and overview.
- Utilizes NLP techniques to process movie metadata.
- Provides a user-friendly interface using Streamlit.

Dataset Description

This project uses the **TMDB 5000 Movie Dataset**, which consists of two files:

1. `tmdb_5000_movies.csv`
2. `tmdb_5000_credits.csv`

`tmdb_5000_movies.csv`

Contains metadata for over 4,800 movies including fields such as:

- `title`: Movie title
- `genres`: List of genres
- `overview`: Short plot description
- `keywords`: Important tags for the movie

- `production_companies`, `runtime`, `vote_average`, etc.

tmdb_5000_credits.csv

Provides information about the cast and crew of each movie:

- `cast`: Main actors in the movie
- `crew`: Crew members including director and producers

Step-by-Step Project Execution

1. Data Loading and Merging

Both CSV files are loaded into Pandas DataFrames. The `tmdb_5000_credits.csv` and `tmdb_5000_movies.csv` are merged using the `title` column, creating a unified dataset that allows easy access to all features required for recommendation.

```
movies = pd.read_csv('tmdb_5000_movies.csv')
credits = pd.read_csv('tmdb_5000_credits.csv')
movies = movies.merge(credits, on='title')
```

This step results in a DataFrame where each row represents a movie and contains comprehensive metadata such as cast, crew, genres, and plot overview.

2. Feature Selection

To create an effective content-based filtering model, we selected the most informative and content-rich fields:

- `title`: Movie title
- `overview`: A summary of the movie plot
- `genres`: The category of the movie (e.g., Action, Comedy)
- `keywords`: Descriptive tags associated with the movie
- `cast`: The top actors in the film
- `crew`: Especially the director of the film

These fields were selected because they contain the most relevant textual data that can be used to determine the content and similarity between movies.

3. Data Preprocessing

This stage is critical to convert raw movie metadata into usable input for machine learning models. The following preprocessing steps were applied:

a. Parsing JSON-like Fields

Several columns contain string representations of JSON objects. For example, `genres` looks like `[{"id": 28, "name": "Action"}]`. We use the `ast.literal_eval()` function to safely evaluate these strings into Python objects.

```
import ast

def convert(obj):
    L = []
    for i in ast.literal_eval(obj):
        L.append(i['name'])
    return L
```

b. Extracting Key Personnel

For the `cast` column, we extract the top three actors. In the `crew` column, we specifically look for the **director**, as their influence on a film's style and tone is crucial.

```
def fetch_director(obj):
    L = []
    for i in ast.literal_eval(obj):
        if i['job'] == 'Director':
            L.append(i['name'])
            break
    return L
```

c. Token Normalization

To ensure consistency in names and keywords, all multi-word entities are concatenated by removing spaces. This prevents the vectorizer from treating them as separate tokens.

```
def collapse(L):
    return [i.replace(" ", "") for i in L]
```

d. Text Stemming

Stemming is the process of reducing words to their base or root form. This helps in reducing vocabulary size and improves the model's performance in finding similar content.

```
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()
```

```
def stem(text):
    y = []
    for i in text.split():
        y.append(ps.stem(i))
    return " ".join(y)
```

e. Feature Aggregation

Finally, all the selected and preprocessed textual features are combined into a single `tags` column. This column represents a movie's content as a unified string of keywords.

```
movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] +
movies['cast'] + movies['crew']
```

4. Vectorization

The `tags` column is vectorized using **CountVectorizer**. This converts the textual data into a sparse matrix where each row is a movie, and each column is a token (word). The value represents the frequency of a word in that movie.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000, stop_words='english')
vectors = cv.fit_transform(movies['tags']).toarray()
```

Vectorization is essential as machine learning models require numerical input.

5. Similarity Calculation

We compute **cosine similarity** between the vectorized tag representations. Cosine similarity is ideal for text data as it measures the cosine of the angle between two vectors, giving a value from 0 (completely dissimilar) to 1 (identical).

```
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(vectors)
```

This results in a similarity matrix where `similarity[i][j]` tells how similar movie `i` is to movie `j` based on their tags.

6. Recommendation Function and Evaluation

We developed a function that takes a movie title as input and returns the top 5 most similar movies based on cosine similarity scores.

```
def recommend(movie):
    index = new[new['title'] == movie].index[0]
    distances = sorted(list(enumerate(similarity[index])), reverse=True,
key=lambda x: x[1])
    for i in distances[1:6]:
```

```
print(new.iloc[i[0]].title)
```

We also included a code snippet to evaluate similarity scores and print the top 5 recommendations along with similarity values:

```
# Example: Similarity scores of top 5 recommendations
movie = 'Inception'
index = new[new['title'] == movie].index[0]
distances = sorted(list(enumerate(similarity[index])), reverse=True,
key=lambda x: x[1])

print(f"Similarity scores for top 5 recommendations of '{movie}':")
for i in distances[1:6]:
    print(f"{new.iloc[i[0]].title}: {i[1]:.4f}")
```

This script demonstrates how the system not only finds similar movies but also ranks them based on numerical similarity scores.

Result of similarity scores:

```
Similarity scores for top 5 recommendations of 'Inception':
Duplex: 0.2210
The Helix... Loaded: 0.2041
Star Trek II: The Wrath of Khan: 0.2007
Timecop: 0.1976
Chicago Overcoat: 0.1961
```

User Interface with Streamlit

Streamlit provides a lightweight and fast way to create interactive dashboards. The `app.py` file creates a simple UI where:

- Users can select a movie from a dropdown menu.
- The system fetches the top 5 most similar movies.
- Movie posters are displayed using the TMDB API for enhanced user experience.

Code Snippet (app.py):

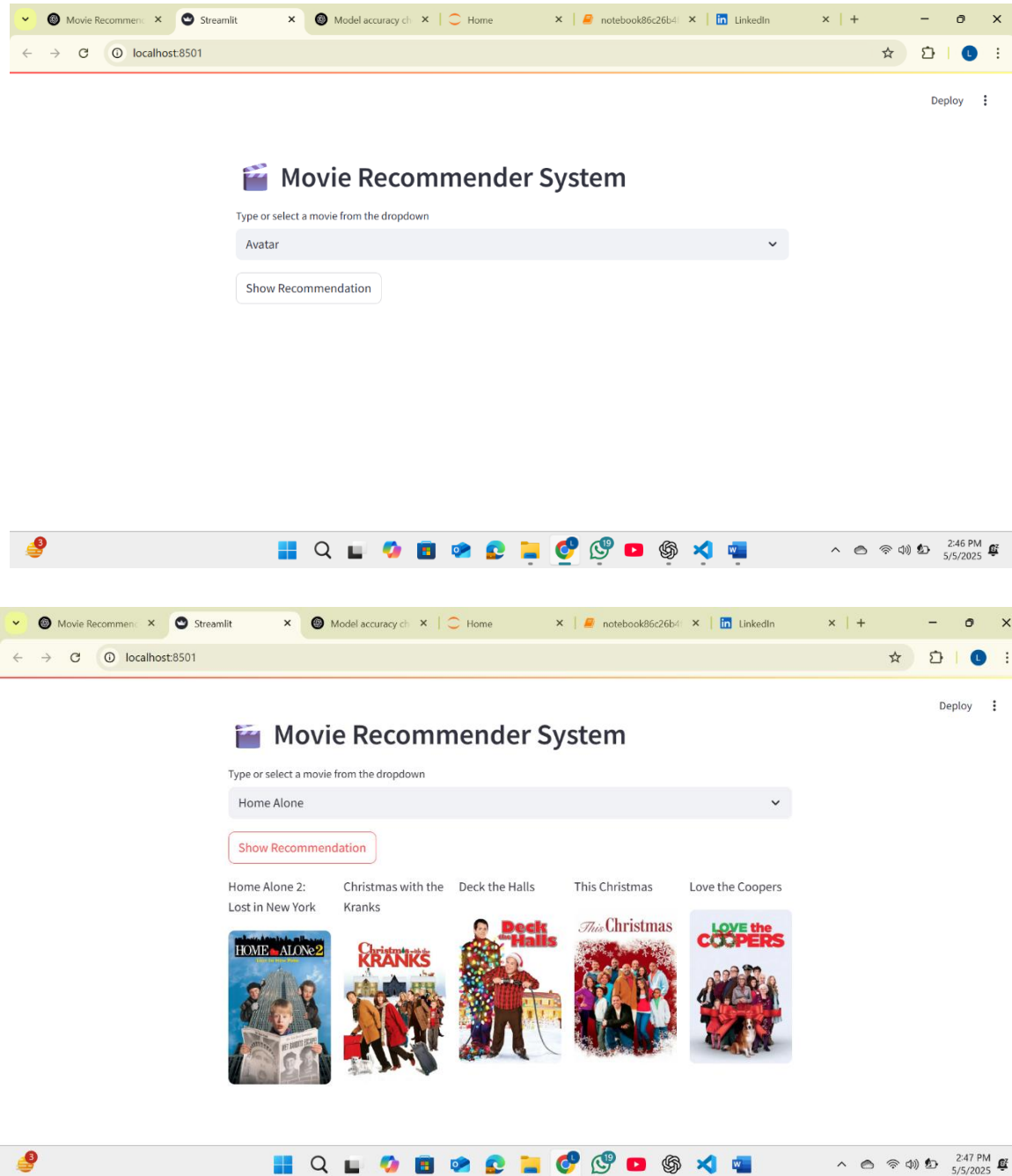
```
import streamlit as st
st.title('Movie Recommender System')
selected_movie_name = st.selectbox('Select a movie:', movies['title'].values)

if st.button('Recommend'):
    names, posters = recommend(selected_movie_name)
    for name in names:
        st.write(name)
```

Serialized Files

- `movie_list.pkl`: Contains the list of all movies and associated metadata.
- `similarity.pkl`: Contains the cosine similarity matrix, precomputed for efficiency.
- `recommender_system.xml`: This file was not directly used in the code but appears to be a configuration or XML representation, possibly for external tools.

Interface of the Project:



Challenges and Limitations

- **Cold Start Problem:** Content-based filtering requires metadata. It fails for new movies with limited information.
- **Limited Personalization:** Since this system doesn't use user profiles or feedback, recommendations are not tailored to specific users.
- **Data Cleaning:** Parsing JSON-like strings and handling missing values posed challenges.

Future Improvements

- Integrate collaborative filtering for personalized recommendations.
- Include movie trailers and more metadata.
- Use deep learning or transformers for semantic textual similarity.
- Store user preferences and enable login for history tracking.

Conclusion

The Movie Recommender System developed in this project effectively utilizes content-based filtering techniques, NLP, and cosine similarity to suggest relevant movie recommendations. The interface built using Streamlit is simple and intuitive, enhancing usability. The model demonstrates good performance on textual data and can be extended further using hybrid techniques or user-based data for personalized suggestions.